

# **Análise experimental e assintótica de algoritmos de ordenação e busca em C++**

Arnaldo José Costa Júnior e Giovanne Isaac Marques<sup>1</sup>

Orientador: Prof. Israel Florentino dos Santos

<sup>1</sup> Faculdade Engenheiro Salvador Arena, Centro Educacional da Fundação Salvador Arena, São Bernardo do Campo - SP

## **Resumo**

Este trabalho apresenta uma análise experimental e teórica de algoritmos clássicos de ordenação e busca implementados em C++. O objetivo é comparar o desempenho prático desses algoritmos com suas respectivas complexidades assintóticas. O estudo baseia-se nos conceitos de análise de algoritmos, complexidade computacional e modelo RAM. A metodologia adotada envolveu a execução de experimentos com diferentes tamanhos de entrada e cenários de dados, incluindo vetores aleatórios, ordenados, reversos e parcialmente ordenados, com 30 repetições por configuração. Foram coletadas métricas como tempo de execução, número de comparações, acessos à memória e profundidade de recursão, além da aplicação de análise estatística com intervalo de confiança de 95%. Os resultados mostram que, embora a análise assintótica descreva corretamente o crescimento dos algoritmos, fatores práticos como cache, implementação e distribuição dos dados influenciam significativamente o desempenho real. Conclui-se que algoritmos  $O(n \log n)$  são mais eficientes para grandes volumes de dados, enquanto algoritmos simples podem ser vantajosos em cenários específicos.

Palavras-chave: algoritmos. ordenação. busca. análise assintótica. desempenho.

## **Abstract**

This work presents an experimental and theoretical analysis of classical sorting and searching algorithms implemented in C++. The objective is to compare the practical performance of these algorithms with their asymptotic complexities. The study is based on concepts such as algorithm analysis, computational complexity, and the RAM model. The methodology involved experiments using different input sizes and data scenarios, including random, sorted, reversed, and partially sorted arrays, with 30 repetitions per configuration. Metrics such as execution time, number of comparisons, memory accesses, and recursion depth were collected, along with statistical analysis using a 95% confidence interval. The results show that, although asymptotic analysis correctly describes algorithm growth trends, practical factors such as cache, implementation, and data distribution significantly impact real performance. It is concluded that  $O(n \log n)$  algorithms are more suitable for large datasets, while simpler algorithms can still be effective in specific cases.

Keywords: algorithms. sorting. searching. asymptotic analysis. performance.

## **1 Introdução**

A escolha de algoritmos eficientes é fundamental para o desempenho de sistemas computacionais. Algoritmos de ordenação e busca são amplamente utilizados em aplicações reais, sendo frequentemente componentes críticos.

A análise assintótica fornece uma estimativa teórica do comportamento dos algoritmos, porém não considera aspectos práticos como hardware e padrões de entrada.

Este trabalho tem como objetivo analisar experimentalmente diferentes algoritmos, comparando seus desempenhos com suas complexidades teóricas.

## **2 Referencial teórico**

A análise de algoritmos é uma área fundamental da ciência da computação que busca avaliar o desempenho de soluções computacionais.

Os algoritmos analisados foram classificados em dois grupos: algoritmos quadráticos ( $O(n^2)$ ) e algoritmos  $O(n \log n)$ . Algoritmos quadráticos apresentam crescimento rápido e tornam-se inviáveis para grandes volumes, enquanto algoritmos  $O(n \log n)$  escalam de forma mais eficiente.

A busca sequencial possui complexidade  $O(n)$ , enquanto a busca binária apresenta  $O(\log n)$ , desde que os dados estejam ordenados.

O modelo RAM assume custo constante para operações básicas, sendo utilizado como base para a análise dos algoritmos.

## **3 Metodologia**

Os experimentos foram realizados em ambiente controlado, utilizando linguagem C++ com compilador otimizado (-O2).

Foram utilizados vetores com tamanhos variando de 1.000 a 1.000.000 elementos, em diferentes cenários: aleatório, ordenado, reverso, parcialmente ordenado, alta repetição e distribuição gaussiana.

Foi utilizada seed fixa para garantir reprodutibilidade.

Foram coletadas métricas como número de comparações, trocas, acessos, tempo de execução, profundidade de recursão e uso de memória.

Cada experimento foi repetido 30 vezes, permitindo análise estatística consistente.

## **4 Resultados e discussão**

Os resultados apresentaram baixo coeficiente de variação para algoritmos determinísticos e maior variabilidade no Quick Sort randomizado.

Os intervalos de confiança foram estreitos, indicando consistência nos experimentos.

Os resultados experimentais confirmaram as previsões teóricas: algoritmos quadráticos apresentaram crescimento proporcional a  $n^2$ , enquanto algoritmos eficientes apresentaram crescimento próximo de  $n \log n$ .

O Bubble Sort mostrou-se inviável devido ao alto custo computacional. O Insertion Sort apresentou excelente desempenho em dados quase ordenados.

O Quick Sort demonstrou sensibilidade à escolha do pivô, enquanto sua versão randomizada reduziu a probabilidade de pior caso.

A análise também evidenciou que fatores como cache e implementação influenciam significativamente o desempenho real.

## 5 Considerações finais

A análise experimental confirmou que a teoria assintótica descreve corretamente o crescimento dos algoritmos, porém não captura todos os fatores práticos envolvidos.

Algoritmos  $O(n \log n)$  são mais adequados para grandes volumes de dados, enquanto algoritmos simples ainda são úteis em cenários específicos.

O estudo reforça a importância de considerar tanto a teoria quanto a prática na escolha de algoritmos.

## Referências

- [1] **TATE, A.** *Comparing Search Algorithms* – CSCI 221. UNC Greensboro, 2025. Disponível em: <https://github.com/atate/COMP-221-Search-Example/blob/master/SearchComparison.cpp>. Acesso em: 20 abr. 2026.
- [2] **MIDDLEBURY COLLEGE.** Sorting I:  $O(n^2)$  sorts – CSCI 0201. Middlebury College, 2020. Disponível em: <https://www.middlebury.edu/college/academics/math>. Acesso em: 20 abr. 2026.
- [3] **MIDDLEBURY COLLEGE.** Sorting II:  $O(n \log n)$  sorts – CSCI 0201. Middlebury College, 2020. Disponível em: <https://www.middlebury.edu/college/academics/math>. Acesso em: 20 abr. 2026.
- [4] **RUNESTONE ACADEMY.** Sorting and Searching Exercises. Runestone Academy, 2021. Disponível em: <https://runestone.academy/>. Acesso em: 20 abr. 2026.
- [5] **CHARBONNIER, J.; HANUSSE, N.; KAO, M.; ROY, S.** Realistic Analysis of some Sorting Algorithms. In: STACS 2013 – 30th International Symposium on Theoretical Aspects of Computer Science. HAL, 2013. Disponível em: <https://hal.archives-ouvertes.fr/>. Acesso em: 20 abr. 2026.
- [6] **CHARBONNIER, J.** *Some Complexity Results about Word Represented Data*. Tese (Doutorado) – HAL Theses, 2014. Disponível em: <https://hal.archives-ouvertes.fr/>. Acesso em: 20 abr. 2026.
- [7] **ST. FRANCIS XAVIER UNIVERSITY.** Searching and Sorting Lab – CSCI 235. St. Francis Xavier University, 2020. Disponível em: <https://people.stfx.ca/>. Acesso em: 20 abr. 2026.
- [8] **QSL.NET.** Sorting Algorithms Educational Material. [QSL.net](http://www.qsl.net), 2019. Disponível em: <http://www.qsl.net/>. Acesso em: 20 abr. 2026.